

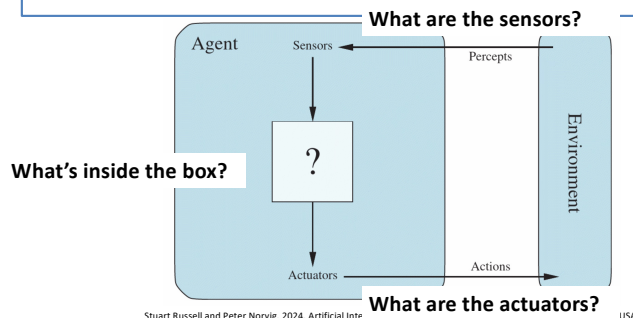
Artificial Intelligence Agents: Core Concepts and Design Patterns for Developing Agentic AI

图文讲义版：用原 PPT 作为视觉锚点，按“概念故事线 + 直觉解释 + 考试速记”整理。

先破 hype：agent 应用空间很大，但定义混乱，必须回到 sensors → decision → actuators 的经典 agent 框架 → 现代落点：LLM-based agent 用文本表示目标、状态、计划和输出，优势是预训练知识与自然语言接口 → 任务循环：Get Mission → Scan Scene → Think Through → Take Action → Learn & Get Better，把聊天变成面向目标的闭环 → 系统坑点：framework 不是 LLM，非文本模态要转表示，上下文有限，长期记忆要外接，动作集合必须被设计和约束 → 为什么要 pattern：agent 行为不像传统程序那样完全固定，battle-tested blueprints 用来提高结构性、可维护性、可靠性和效率 → 从简单到复杂：single-call 适合一步题；prompt chaining 顺序拆解；routing 按条件选路径；parallelisation 并行做独立子任务 → 真正接世界：tool use 让模型调用数据库、API、代码或 RAG；MCP 则把这种外部通信标准化成可扩展协议 → 考试总收束：高分答案要比较模式 trade-off，并指出 agent reliability/safety 难在多步规划、上下文、工具执行和 action space 控制

Did we need a new definition?

"An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators."
-- Russell and Norvig



Alessandro Suglia

ATNLP Week 10/ Lecture 28

School of Informatics

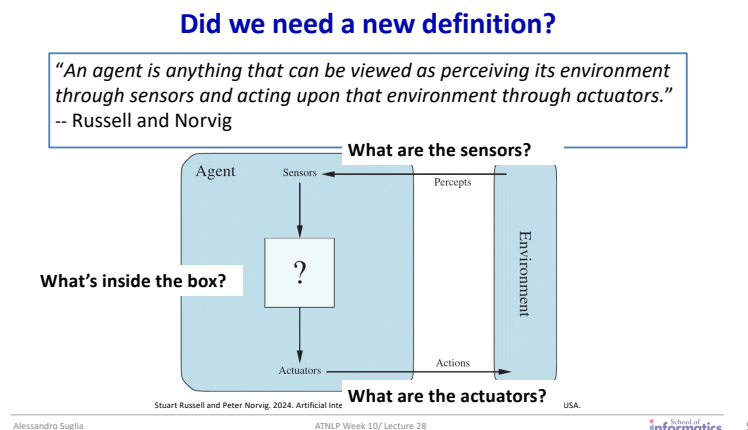
9

原 PPT 第 9 页

这节课一句话

这节课一句话：AI agent 不是“会聊天的 LLM 加一个时髦外壳”，而是一个在环境中感知、决策、行动的系统；LLM 只是它的大脑候选件，真正决定能不能可靠落地的是信息流、action space、memory/context 管理，以及 prompt chaining、routing、parallelisation、tool use、MCP 这些设计模式怎样把不稳定的语言模型变成可控工作流。

1. Agent 定义：别被 hype 带跑，先问 sensors 和 actuators 在哪



原 PPT 第 9 页

人话直觉

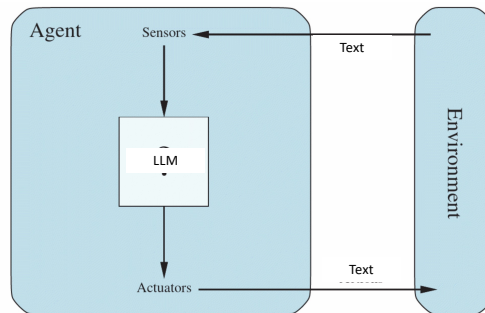
把 agent 想成一个替你办事的实习生，而不是一个会聊天的网页框。真正的问题不是“它是不是用了 LLM”，而是：它看见了什么？能记住什么？能做什么？做完以后环境有没有变化？如果一个系统只能生成一段文本，它更像答题机；如果它能观察环境、选择动作、执行并接收反馈，才开始像 agent。

精确定义 / 机制：Russell and Norvig 的经典定义把 agent 描述为：通过 sensors perceiving its environment，并通过 actuators acting upon that environment 的任何东西。现代 McKinsey、Gov.UK 等定义会强调代表用户执行任务、协调流程、自主或合作地达成目标，但这些说法本质上仍能落回感知、决策、行动三件事。对 LLM agent 来说，sensors 可以是用户输入、检索结果、网页、数据库、工具 observation 或多模态输入；actuators 可以是生成回复、调用 API、写文件、点击 GUI、运行代码或发出控制命令。考试时要避免把 agent 简化成“LLM + prompt”：LLM 只是 box 里面可能负责 reasoning/decision 的组件，agent 的边界还包括外部环境接口、状态管理和动作空间。

考试问法：如果考“define AI agent / why modern definitions still relate to classical AI”，答案先写 Russell & Norvig：sensors 感知环境，actuators 作用环境；再解释 LLM-based agent 是这个框架在语言和工具环境中的实现；最后补一句现代定义强调 agency、代表用户执行任务、协调 workflow 和目标达成，但没有推翻经典定义。

2. LLM-based agent anatomy: LLM 是大脑，不是整个身体

The Anatomy of an LLM-based AI Agent



Main advantages: 1) leverage pretrained knowledge, 2) uses text to represent inputs/outputs

Alessandro Suglia

ATNLP Week 10 / Lecture 2B

School of Informatics

10

原 PPT 第 10 页

人话直觉

这页可以想成给 agent 画 X 光片：中间那块 LLM 很亮眼，但它不是手、不是眼睛、也不是记忆本。LLM 的超能力是把很多东西翻译成文字来理解和生成；它的脆弱点也在这里——如果世界信息进不来、动作执行不出去、状态保存不住，再聪明的大脑也只能原地空想。

精确定义 / 机制： LLM-based AI agent 的最小 anatomy 是 LLM 接收 text representation，并输出 text representation。它的主要优势有两点：第一，能够 leverage pretrained knowledge，把大规模预训练中获得的语言、常识、代码和任务知识用于决策；第二，natural language 可以统一表示目标、约束、计划、工具描述、工具结果和最终答案。这个 text interface 让 agent 开发变得灵活：用户可以用自然语言给 mission，框架可以把工具 schema、RAG 文档或环境 observation 编码进 prompt，模型再生成计划或结构化调用。但注意，语言接口不是魔法接口；图像、音频、GUI、数据库行、API schema 都必须先被包装成模型能处理的表示，输出也必须被解析成安全可执行的动作。

考试问法：如果考“components/advantages of LLM-based agents”，写 LLM、textual input/output representation、wrapper/framework、可选 tools、memory 和 workflow control。优势写 pretrained knowledge 和自然语言表示；限制顺手引到 modality gap、context length、lack of memory 与 action space。

3. Agentic problem-solving process: 从聊天框变成任务闭环

Agentic AI Problem-Solving Process

- **Get the Mission:** Use natural language to specify goals and constraints
- **Scan the Scene:** The agent can autonomously determine the relevant sources of information required (e.g., RAG)
- **Think it Through:** Use reasoning steps to determine the best plan given the information available
- **Take Action:** Generate an action that will bring the agent closer to the goal
- **Learn & Get Better:** Learn from the interaction with the environment to solve tasks better (ideally!)

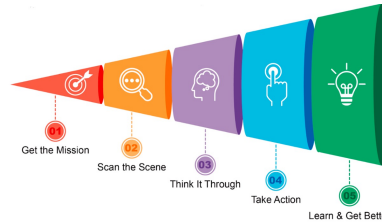


Fig. 1 Agentic AI functions as an intelligent assistant, continuously learning through experience. It operates via a straightforward five-step loop to accomplish tasks

原 PPT 第 11 页

人话直觉

普通聊天像你问一句、它答一句；agentic process 更像派一个助教完成任务。你先交代任务和限制，它去看资料，想方案，采取行动，再根据反馈改进。真正的 agent 不是把答案吐出来就结束，而是在“目标—信息—计划—行动—反馈”之间转一圈。

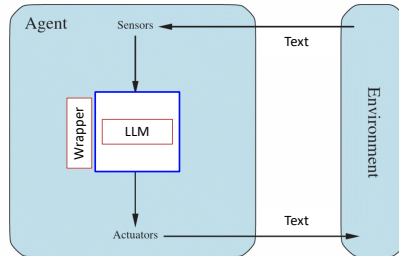
精确定义 / 机制： Lecture 28 用五步描述 agentic AI problem-solving process。Get the Mission 是用自然语言指定目标和约束，例如“只选经伦敦直飞的航班”。Scan the Scene 是 autonomously determine relevant information sources，例如调用 RAG、搜索文档、查数据库。Think it Through 是使用 reasoning steps 根据已有信息制定计划。Take Action 是生成让系统更接近目标的动作，例如调用 booking API、写邮件或更新记录。Learn & Get Better 是理想情况下从环境交互中学习，至少利用反馈修正后续行为。Slide 12 还强调，这个话题串起本课程前面知识：LLMs、prompting、RAG、long-context modelling 和 multimodal，因为 agent 同时需要语言理解、信息获取、长程状态和可能的多模态环境接入。

考试问法：如果考“list and explain the agentic problem-solving process”，不要只背五个英文短语。每个短语配功能：Mission= 目标/约束，Scene= 信息源/RAG，Think= 计划/推理，Action= 执行动作，Learn= 反馈改进。最后说明 agent 结合 prompting、RAG、long context 和 multimodal。

4. Design pitfalls: agent 最容易坏在接口、记忆和动作边界

Pitfalls in Designing AI Agents

- **Framework $\neq$ LLM**: you never interact with the LLM but with a wrapper around it!
- **Modality gap**: any input source of data needs to be converted into text to make sure that can be used by the LLM
- **Context length limit**: LLMs are inherently limited by the number of tokens that they can process
- **Lack of memory**: LLMs do not have notions of long-term memory (unless simulated by RAG)
- **Action space**: many agentic frameworks assume a specific action space that your LLM can select actions from



原 PPT 第 13 页

人话直觉

一个 agent 像一辆自动驾驶车：发动机不错不代表整车安全。传感器读错、地图太短、记忆丢失、方向盘权限过大，都会出事。LLM agent 也是一样，最危险的常常不是它不会生成句子，而是系统误以为“会生成句子”就等于“能可靠行动”。

精确定义 / 机制：这页列出五个常见 pitfall。第一，Framework $\neq$ LLM：用户通常和 wrapper/framework 交互，不是直接和裸模型交互，因此 prompt 注入、工具解析、错误处理和权限都发生在包装层。第二，Modality gap：任何非文本输入都需要转换成 LLM 可用表示，转换会损失信息或引入噪声。第三，Context length limit：任务历史、工具结果、RAG 文档和系统指令都会占 token，长任务容易截断、遗忘或成本爆炸。第四，Lack of memory：LLM 本身没有真正长期记忆，长期状态通常要靠 RAG、数据库、日志或外部 memory 模拟。第五，Action space：框架预设模型可选动作集合；动作集合越强，越需要约束、验证和安全防护。

考试问法：如果考“pitfalls in designing AI agents”，按五点展开并各给影响。高分补一句：agent reliability 难是因为这些问题会级联，例如 router 误判导致错误工具，错误 observation 进入上下文，再触发错误 action。

5. Prompt chaining: 把大任务切成可检查的小工序

Pattern 1: Prompt Chaining

- **Prompt chaining** or also called **Pipeline** pattern represents a powerful design pattern for handling complex tasks
- The core idea is to break down the original problem into a sequence of smaller and more manageable sub-problems (*divide-et-impera*)
- We can create a sub-module with a custom prompt for each sub-problem
- And then, the output generated from one prompt is strategically fed as input into the next module in the chain

原 PPT 第 17 页

人话直觉

Prompt chaining 像流水线做报告：第一位同学只负责总结材料，第二位只负责抽趋势，第三位只负责写邮件。每个人工作更简单，老师也更容易检查哪一步错了；但如果第一位把报告总结歪了，后面的人会很认真地把错误继续加工成漂亮成品。

精确定义 / 机制： Prompt chaining，也叫 pipeline pattern，用 divide-and-conquer 处理复杂任务。开发者把原始任务拆成一串更小、更可管理的 sub-problems，为每个子问题写 custom prompt 或 sub-module，然后把前一步 output strategically fed as input into the next module。它适合多阶段但顺序依赖明确的任务，例如 slide 18 的市场报告例子：先 summarise key findings，再 identify trends and data points，最后 draft email。优点是 modularity、clarity、ease of debugging 和通常更高 reliability；缺点是 inefficient，因为多次调用增加延迟和成本；inability to keep track of global context；error propagation；以及模块之间必须有严格输入输出约束。

考试问法： 如果考“prompt chaining strengths and weaknesses”，答案结构用 definition → example → pros/cons。核心句：chaining improves complex task handling by decomposing into sequential modules, but errors and context loss can propagate through the chain.

6. Routing: 给 agent 加一个分诊台和安全闸门

Routing Implementations

- Routing introduces conditional logic which allows more flexible and context-aware system behaviour
- Routing determines the **user intent** and triggers the correct **behaviour**
- Routing can be implemented in several ways:
 1. **Rule-based routing**: uses predefined rules based on keywords, patterns or structured data extracted from the input
 2. **ML Model-based routing**: it employs a discriminative model (e.g., classifier) trained to classify the request of the user (intent classification)
 3. **Embeddings-based routing**: The input query can be converted into a vector (as in RAG) and compared to embeddings representing different routes or capabilities
 4. **LLM-based routing**: The language model itself can be prompted to analyse the input and output a specific intent label

原 PPT 第 21 页

人话直觉

Routing 就像医院分诊：发烧、骨折、心理咨询不能都排到同一个窗口。agent 也一样，订机票、回答知识题、危险请求、参数缺失请求应该走不同路径。一个好 router 让系统显得聪明；一个坏 router 会把病人送错科室，后面再强的医生也救不回来。

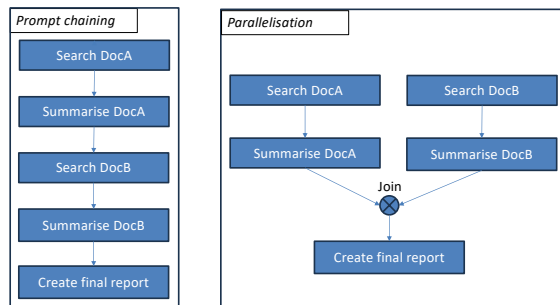
精确定义 / 机制： Routing pattern 超越 prompt chaining 的固定顺序，引入 conditional logic，让系统根据 user intent、输入类型、约束或安全风险触发正确行为。Slide 21 给出四种 implementation。Rule-based routing 使用关键词、模式或结构化字段，简单可控但不灵活。ML model-based routing 使用 discriminative classifier 做 intent classification，适合标签稳定的业务。Embeddings-based routing 把 query 转成向量，与不同 route/capability 的 embedding 比相似度，和 RAG 思路相近。LLM-based routing 让语言模型分析输入并输出 intent label，灵活但也可能不稳定。Routing 的 safety 价值在于可以先做 guardrail，例如 slide 22 的 flight booking 例子先检查 direct connections via London 等 constraint，再抽取参数、规划预订、生成确认邮件。

考试问法： 如果考“routing and its implementations”，要写 routing = conditional selection of workflow/component；列四种实现；优点是 dynamic/reactive、能处理多种输入并送到最佳组件；缺点是 router 本身成为 failure point，misclassification 会导致错误流程或安全问题。

7. Parallelisation: 能同时做的事就同时做，但 join 不是免费午餐

Parallelisation – Multi-document Summarisation

Task: Search DocA and DocB, then generate a summary that includes the main facts from both documents as well as important differences.



Alessandro Suglia

ATNLP Week 10 / Lecture 2B

School of Informatics

25

原 PPT 第 25 页

人话直觉

如果你要比较两篇文档，chaining 像一个人先读 A、总结 A、再读 B、总结 B；parallelisation 像两个人同时读 A 和 B，最后坐在一起合并结论。速度快了，但最后开会很关键：重复事实要去掉，冲突说法要处理，谁卡住了也不能让整份报告失控。

精确定义 / 机制：Parallelisation pattern 适合可拆成 independent sub-problems 的复杂任务。与 prompt chaining 的顺序依赖不同，parallel execution 让多个子任务同时运行，从而减少总等待时间。Slide 25 的 multi-document summarisation 展示了典型 fork/join：Search DocA 与 Search DocB 可以并行，Summarise DocA 与 Summarise DocB 也可并行，然后 Join，最后 Create final report。优点是 speeds-up processing of independent tasks、avoid sequential calls；代价是 fork 时 computational load 更高，join 时有合并 overhead，还可能遇到某个 sub-agent stuck、不同结果冲突或粒度不一致。生产系统里 join step 往往要做去重、冲突解决、置信度比较和最终格式控制。

考试问法：如果考“parallelisation vs prompt chaining”，直接对比：chaining = sequential, output of one step feeds next；parallelisation = independent subtasks run simultaneously then join。再写优点是快，缺点是计算负载和合并复杂度。

8. Tool use: 让 LLM 从会说话变成会办事

Tool Use Pattern – how does it work?

- The Tool Use Pattern allows an LLM to decide when and how to use a specific external function based on the current state (e.g., the user's request intent)
- This pattern is easy to implement because it relies on the ability of LLMs to generate code (typically in Python):
 - Tool definition: external functions and services are defined and described to the LLM
 - LLM decision: the LLM receives the tool definitions as well as the user request as input, and decides which tools should be invoked
 - Function Call Generation: if the LLM decides to use a tool, it generates a structured output (e.g., JSON object or Python) that specifies the name of the tool to call and the arguments (i.e., parameters) to pass to it
 - Tool execution: The agentic framework retrieves the referenced tool and executes it
 - Observation/Result processing: We reformat the output for the LLM
 - LLM processing (Optional): The LLM receives the tool output as context and uses it to generate the final answer -> this fits very well with another pattern; which one?

原 PPT 第 35 页

人话直觉

LLM 本身像一个知识丰富但坐在房间里的顾问：它可以建议查天气、算价格、订航班，但不能凭空接触真实系统。Tool use 给它一组带说明书的遥控器：什么时候按哪个按钮、填什么参数、拿回什么结果，都必须结构化，否则遥控器按错就会出事故。

精确定义 / 机制： Tool Use Pattern 允许 LLM 根据当前状态和用户意图决定 when and how to use an external function。基本流程是：Tool definition 先把外部函数、数据库或服务的名称、描述、参数 schema 和约束提供给 LLM；LLM decision 阶段模型读用户请求和工具定义，判断是否需要工具；Function call generation 阶段输出结构化调用，例如 JSON object 或 Python code，指定 tool name 与 arguments；Tool execution 由 agentic framework 找到对应工具并执行；Observation/result processing 把工具输出整理成模型能理解的上下文；最后可选 LLM processing，用工具结果生成最终答案。它的优势是访问最新数据、扩展 LLM 功能、能和 chaining/routing/parallelisation 结合；缺点是增加 inference-time tokens，要求模型可靠生成可执行结构化输出，并引入工具失败、参数错误和安全风险。

考试问法：如果考“describe the tool use pattern”，按六步流程答最稳。若问 tool use 和 RAG 的关系：RAG 是一种检索外部信息的特定工具/模式；tool use 更广，还包括数据库、API、代码执行、搜索、业务系统操作等。

9. MCP: tool use 是会用工具，MCP 是大家用同一种插座

Model Context Protocol (MCP)

- Using language to represent inputs and actions is an effective way to leverage the existing knowledge contained in LLMs
- However, to interact with real-world systems (e.g., databases), LLMs need to use very specific commands and interfaces
- MCP != Tool use: MCP represents an open standard designed to standardise how LLMs can communicate with external applications, data sources, and tools
- MCP can be intended as a general-purpose communication protocol that allows LLMs to obtain context, execute actions, and interact with various systems
- MCP is now considered an open standard that can be found here: <https://github.com/modelcontextprotocol/modelcontextprotocol>

原 PPT 第 38 页

人话直觉

如果每个工具都有自己的插头，agent 每接一个系统都要重新学一套接口，企业环境会变成一团线。MCP 的直觉就是统一插座：数据库、文件、工具、prompt、resource 都按标准方式暴露，LLM 应用不必为每个外部系统重新发明通信协议。

精确定义 / 机制： Model Context Protocol (MCP) 是一个开放标准，用来 standardise how LLMs communicate with external applications, data sources, and tools。它不等同于 tool use：tool use 是 agent 调用外部函数的设计模式；MCP 是支持这种交互的 general-purpose communication protocol，让 LLM 获得 context、执行 actions，并与多种系统交互。Slide 40 进一步说明，MCP 适合 complex、scalable、enterprise-grade agentic systems，尤其是需要在 heterogeneous and distributed services 组成的生态中运行时。MCP 使用 client-server model 标准化 information flow，并支持 prompts、resources、tools 等组件定义。考试里它常和 tool use 对比：tool use 讲“模型决定调用什么工具”，MCP 讲“工具、资源和上下文如何用标准协议暴露给 agent”。

考试问法：如果考“MCP vs tool use”，一句话区分：tool use is a pattern for invoking external functions; MCP is an open protocol standardising communication with external apps, data sources and tools. 再补企业级重要性：异构分布式服务需要统一信息流和接口。

10. 总比较: agent 设计题要答模式, 也要答 trade-off

Summary

- Agentic AI is becoming an important topic of interest for researchers and practitioners. A lot of hype around it which we need to carefully navigate.
 - Different frameworks have been developed
 - Different definitions have been proposed
 - Ultimately, they all boil down to Russell&Norvig definition!
- When defining an agentic AI application, it's important to consider the information flow and when specialised decision making is required
- For this reason, many different "design patterns" are possible and they should be used to maximise efficiency and efficacy
- Still a lot to be done in this space to ensure reliability and safety of agentic AI applications
- Next lecture: can we go beyond agents that only process text?

Alessandro Suglia

ATNLP Week 10 / Lecture 28

School of Informatics

44

原 PPT 第 43 页

人话直觉

考试里的 agent 题最怕写成“agent 很自主, 很智能, 很有未来”。高分答案像工程评审: 这个任务是一口气 single-call 就够, 还是要 chain? 需不需要 router 防危险请求? 哪些步骤能 parallel? 哪些地方必须 tool use? action space 怎么限制? 哪里会出错?

精确定义 / 机制: Lecture 28 的 summary 强调: agentic AI 热度高、定义多、框架多, 但最终仍回到 Russell & Norvig 的 agent 定义。构建 agentic AI application 时, 要考虑 information flow 以及何时需要 specialised decision making。不同 design patterns 是为了 maximise efficiency and efficacy: single-call 简单、便宜、易测, 但不适合复杂多步任务; prompt chaining 提高模块化和调试性, 但有延迟、上下文和错误传播; routing 让系统动态选择路径并加入 safety guardrails, 但 router 误分类会成为单点故障; parallelisation 加速独立任务, 但需要 join/merge; tool use 扩展现实能力, 但带来结构化输出、执行错误、token 成本和权限风险; MCP 则帮助在大规模系统中标准化外部通信。可靠性和安全仍是开放问题, 因为 agent 的错误可能跨规划、检索、上下文、工具和动作执行层层放大。

考试问法: 如果考综合比较题, 用表格式语言组织: pattern → when useful → strength → weakness。最后收束到 reliability/safety: agent 能执行真实动作, 所以必须限制 action space、验证参数、记录状态、处理工具失败, 并设计安全路由和人类监督。

考试速记版

- Agent 经典定义: 通过 sensors 感知环境, 通过 actuators 作用环境; 现代 LLM agent 仍可落回这个框架。
- LLM-based agent 的优势: 利用 pretrained knowledge, 并用 text 表示目标、约束、计划、工具结果和输出。
- Agentic process: Get Mission → Scan Scene → Think Through → Take Action → Learn & Get Better。

- Agent 话题综合了 LLMs、prompting、RAG、long-context modelling 和 multimodal modelling。
- Framework != LLM: 用户交互的是 wrapper/framework, 工具解析、权限、安全和错误处理都在包装层。
- Modality gap: 非文本数据必须转换成 LLM 可用表示, 转换会损失信息或引入噪声。
- Context length limit 与 lack of memory: 长任务历史会占 token, 长期状态通常要靠 RAG、数据库或外部 memory。
- Action space 决定 agent 能做什么; 能力越强, 越需要权限限制、参数验证和安全 guardrails。
- Single-call execution: 一次 prompt 得到输出, 适合简单一步任务, 不适合复杂多步问题。
- Prompt chaining: 顺序拆解复杂任务, 优点是模块化、清晰、易调试、可靠; 缺点是低效、上下文跟踪难、错误传播。
- Routing: 用条件逻辑按 intent/输入/安全风险选择流程; 实现包括 rule-based、ML classifier、embeddings、LLM-based。
- Parallelisation: 独立子任务并行执行再 join, 优点是快, 代价是 fork 计算负载和合并 overhead。
- Tool use: LLM 根据工具定义生成结构化函数调用, 执行后把 observation/result 放回上下文。
- MCP: 不是 tool use 本身, 而是标准化 LLM 与外部应用、数据源、resources、tools 通信的开放协议。
- 综合比较模板: single-call 简单; chaining 可调试但会传播错误; routing 灵活但会误分类; parallelisation 快但难合并; tool use 强大但有执行风险。

一段稳妥考场回答

An AI agent can be defined in the classical Russell and Norvig sense as a system that perceives its environment through sensors and acts upon that environment through actuators. Modern LLM-based agents are an implementation of this idea in a language and tool environment: the LLM uses text to represent goals, context, plans, tool definitions and outputs, while wrappers, memories, tools and workflows provide the surrounding agent body. A typical agentic problem-solving loop is Get the Mission, Scan the Scene, Think it Through, Take Action, and Learn & Get Better. Designing such systems is difficult because the framework is not the LLM itself, non-text inputs create a modality gap, context length is limited, long-term memory must usually be simulated externally, and the action space must be carefully constrained for safety. Agentic design patterns address these issues. Single-call execution is simple and testable for one-step tasks but does not scale well to complex problems. Prompt chaining decomposes a task into sequential submodules, improving modularity and debugging, but can be inefficient and propagate early errors. Routing introduces conditional logic so that different intents, constraints or safety cases go to different components, but the router can misclassify. Parallelisation runs independent subtasks simultaneously and then joins their results, reducing latency but increasing compute and merge complexity. Tool use lets the LLM decide when and how to call external databases, APIs, code or RAG tools via structured outputs, extending its capabilities but adding token cost, execution failures and security risks. MCP should be distinguished from tool use: tool use is the pattern of invoking external functions, whereas the Model Context Protocol is an open standard for communication between LLM applications and external tools, resources and data sources. Overall, reliable agent design requires not just a strong LLM but explicit information flow, validated interfaces, constrained actions and careful trade-offs between efficiency, flexibility, maintainability and safety.